

**INSTITUTO
FEDERAL**
Santa Catarina

Modularização e Organização de Dependências

Programação Frontend II

Análise e Desenvolvimento de Sistemas

Prof. Adriano Lima

adriano.lima@ifsc.edu.br



cores

#111027

#277756

#16ABCD

#FFF4EC

#C74E23

fontes

Fira Sans Extra Condensed

Ubuntu

Roboto Mono



Projetos Frontend

histórico do uso do JS

■ anos 90 – início dos anos 2000

- js visto como complemento ao html e css
- efeitos visuais simples
- trechos embutidos diretamente no html
- um único arquivo em projetos maiores

■ problemas

- dificuldade de manutenção
- conflitos entre variáveis e funções globais
- quase sem reaproveitamento de código

■ anos 2000

- js arquivos externos (script.js)
 - incluídos no html com `<script src="...">`
- menos poluição visual
- reutilização em diferentes páginas

■ limitações

- tudo em um único arquivo gigante
- crescimento do código dificultava a manutenção



Projetos Frontend

histórico do uso do JS

■ 2005 – 2010

- aplicações mais dinâmicas (web 2.0)
- bibliotecas como jQuery, Dojo, YUI
- IIFE: escopo próprio, evitando poluição global
- Namespaces: um único objeto global

■ problemas

- cada biblioteca com seu jeito de modularizar
- faltava um padrão oficial para o ecossistema

■ 2010 – 2015

- CommonJS (Node.js): backend
 - uso de `require` e `module.exports`
- AMD (Asynchronous Module Definition):
 - ~~front~~ usava `define` e `require`
- bundlers surgem (Browserify e Webpack)

■ problemas

- dois padrões diferentes competindo
- sintaxe complexa para o navegador



Projetos Frontend

histórico do uso do JS

■ 2015 – atualidade

- padronização dos módulos com ES6
 - fornece import e export

■ benefícios

- padrão oficial
- melhor legibilidade e manutenção
- funciona no navegador e no node

■ impacto

- padrão para frameworks modernos
- facilitou integração com ts e bundlers



ES6 MODULES

Projetos Frontend

modularização

- divisão do sistema em partes menores
- facilita manutenção e evolução do projeto
- evita duplicação de código
- melhora a colaboração em equipe
- aumenta a reutilização de componentes

ES6

- cada arquivo é um módulo
- variáveis, funções e classes são:
 - privados dentro do módulo
 - têm que ser explicitamente exportados
 - tem que ser explicitamente importados
- sintaxe diferente dos módulos node
- pode ser usado no frontend e no backend
- adicionado ao html com
`<script type="module">`



Projetos Frontend

exportação

- exportação nomeada
 - apenas a palavra export
 - inline ou ao final do arquivo
 - vários elementos de um mesmo arquivo
- exportação *default*
 - com as palavras export default
 - inline ou ao final do arquivo
 - apenas um elemento principal por arquivo
- combinar exportações
 - 1 exportação *default*
 - 1+ exportações nomeadas

exemplos

```
// exportação nomeada
export function soma(a, b) { return a + b }
export function subtrai(a, b) { return a - b }

function primeiro(array) { return array[0] }
function ultimo(array) { return array[array.length - 1] }

export { primeiro, ultimo }

// exportação default
export default class Usuario {
  constructor(nome) {
    this.nome = nome;
  }
}

function multiplica(a, b) { return a + b }

export default multiplica;
```



Projetos Frontend

importação

- importação nomeada
 - apenas a palavra `import`
 - importar com apelidos (*alias*)
 - importar tudo de um módulo
- importação *default*
 - apenas a palavra `import`
 - qualquer nome
- combinar importações
 - não há limite

exemplos

```
// importação nomeada
import { soma, subtrai } from './utils/math.js';

console.log(soma(10, 5))
console.log(subtrai(10, 5))

import { upper as paraMaiusculo } from
'./utils/string.js';

console.log(paraMaiusculo("ifsc"))

import * as constantes from './utils/constantes.js';

console.log(constantes.PI)
console.log(constantes.APP_NAME)

// importação default
import multiplica from './utils/multiplica.js';

console.log(multiplica(5, 7));
```



Projetos Frontend

boas práticas de modularização

- apenas uma responsabilidade por módulo
- organizar pasta por domínio
 - components/, pages/, services/, utils/
- utilizar nomes claros
 - evitar nomes genéricos (index2.js ou teste.js)
- agrupar funções relacionadas
- reutilizar código (DRY)
 - criar bibliotecas internas de funções comuns



Projetos Frontend

iniciando um projeto

- **pré-requisito** node.js com npm
 - alternativas yarn ou pnpm
- criar diretório do projeto
- inicializar o `package.json`
 - interativo
 - automático
- instalar dependências iniciais
 - de execução
 - de desenvolvimento

exemplos

```
node -v
npm -v

mkdir meu-projeto-frontend
cd meu-projeto-frontend

npm init

npm init -y

npm install --save axios

npm install -D eslint
```



Projetos Frontend

configurando um projeto

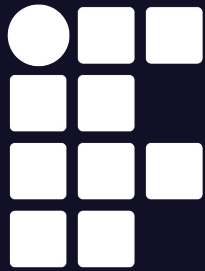
- arquivo `package.json`
 - identifica o projeto
 - gerencia dependências
 - padroniza scripts
 - garante reprodutibilidade
 - centraliza configurações
- atualizar as dependências ao clonar
 - `npm install` ou `npm i`

exemplo

`package.json`

```
{
  "name": "meu-projeto-frontend",
  "version": "1.0.0",
  "description": "Um projeto de exemplo em frontend",
  "author": "Adriano Lima",
  "license": "MIT",
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "test": "jest"
  },
  "dependencies": {
    "react": "^18.0.0",
    "axios": "^1.7.0"
  },
  "devDependencies": {
    "vite": "^5.0.0",
  }
}
```





**INSTITUTO
FEDERAL**
Santa Catarina

Modularização e Organização de Dependências

Programação Frontend II

Análise e Desenvolvimento de Sistemas

Prof. Adriano Lima

adriano.lima@ifsc.edu.br

